CS 4375 HW3

Shivesh Gupta

October 2026

1 Theoretical Part

1.1 Softmax and Cross-Entropy

a. With $\mathbf{z} = (2, 1, 0)$, first find the exponentials and their sum.

$$\begin{aligned} e^2 &= 7.3891 \\ e^1 &= 2.7183 \\ e^0 &= 1.0000 \\ \text{sum} &= 11.1073 \end{aligned}$$

Then divide each one by the sum.

$$\begin{aligned} y_1 &= \frac{7.3891}{11.1073} = 0.6652 \\ y_2 &= \frac{2.7183}{11.1073} = 0.2447 \\ y_3 &= \frac{1.0000}{11.1073} = 0.0900 \end{aligned}$$

Check: $0.6652 + 0.2447 + 0.0900 = 1.0000$.

b. The correct class is class 1, so only y_1 matters.

$$\begin{aligned} L &= -\ln y_1 \\ &= -\ln 0.6652 \\ &= 0.4076 \end{aligned}$$

If the network predicted $y_1 = 1$ exactly, then

$$\begin{aligned} L &= -\ln 1 \\ &= 0 \end{aligned}$$

which is the smallest loss possible.

c. Using $\partial L / \partial z_k = y_k - t_k$:

$$\begin{aligned} \frac{\partial L}{\partial z_1} &= 0.6652 - 1 = -0.3348 \\ \frac{\partial L}{\partial z_2} &= 0.2447 - 0 = 0.2447 \\ \frac{\partial L}{\partial z_3} &= 0.0900 - 0 = 0.0900 \end{aligned}$$

Gradient descent subtracts the gradient, so z_1 goes up (its gradient is negative) and z_2 and z_3 go down (their gradients are positive). This makes sense. We want the correct class to get a bigger score and the wrong classes to get smaller scores.

d. e^{1002} is about 10^{435}, which is far bigger than the largest number a double can hold ($\approx 1.8 \times 10^{308}$). The computer returns `inf`, and `inf/inf` gives `NaN`.

To fix this, divide the top and bottom by e^{1000}:

$$y_1 = \frac{e^{1002}/e^{1000}}{(e^{1002}+e^{1001}+e^{1000})/e^{1000}} = \frac{e^2}{e^2+e^1+e^0} = 0.6652$$

$$y_2 = \frac{e^{1001}/e^{1000}}{(e^{1002}+e^{1001}+e^{1000})/e^{1000}} = \frac{e^1}{e^2+e^1+e^0} = 0.2447$$

$$y_3 = \frac{e^{1000}/e^{1000}}{(e^{1002}+e^{1001}+e^{1000})/e^{1000}} = \frac{e^0}{e^2+e^1+e^0} = 0.0900$$

These are the same numbers as in part (a). In practice, code subtracts the largest logit before taking exponentials for exactly this reason.

1.2 One Step of Backpropagation by Hand

Setup: $\mathbf{x} = (1, 2)$, $t = 1$, $\eta = 0.1$.

a. **Forward pass.**

$$z_3 = w_{31}x_1 + w_{32}x_2 + b_3 = 0.5(1) - 0.25(2) + 0.25 = 0.25$$
$$a_3 = \mathrm{ReLU}(0.25) = 0.25$$

$$z_4 = w_{41}x_1 + w_{42}x_2 + b_4 = -1(1) + 0.25(2) - 0.5 = -1$$
$$a_4 = \mathrm{ReLU}(-1) = 0$$

$$z_5 = w_{53}a_3 + w_{54}a_4 + b_5 = 2(0.25) + 1(0) - 0.5 = 0$$
$$y = \sigma(0) = 0.5$$

$$L = -\big[1 \cdot \ln 0.5 + 0\big] = 0.6931$$

b. **Backward pass.** First the deltas.

$$\delta_5 = y - t = 0.5 - 1 = -0.5$$
$$\delta_3 = \delta_5\, w_{53}\, \mathrm{ReLU}'(z_3) = (-0.5)(2)(1) = -1$$
$$\delta_4 = \delta_5\, w_{54}\, \mathrm{ReLU}'(z_4) = (-0.5)(1)(0) = 0$$

Now the gradients, using $\partial L/\partial w_{ji} = \delta_j a_i$ and $\partial L/\partial b_j = \delta_j$.

$$\frac{\partial L}{\partial w_{31}} = \delta_3 x_1 = (-1)(1) = -1$$
$$\frac{\partial L}{\partial w_{32}} = \delta_3 x_2 = (-1)(2) = -2$$
$$\frac{\partial L}{\partial b_3} = \delta_3 = -1$$

$$\frac{\partial L}{\partial w_{41}} = \delta_4 x_1 = 0$$
$$\frac{\partial L}{\partial w_{42}} = \delta_4 x_2 = 0$$
$$\frac{\partial L}{\partial b_4} = \delta_4 = 0$$

$$\frac{\partial L}{\partial w_{53}} = \delta_5 a_3 = (-0.5)(0.25) = -0.125$$
$$\frac{\partial L}{\partial w_{54}} = \delta_5 a_4 = (-0.5)(0) = 0$$
$$\frac{\partial L}{\partial b_5} = \delta_5 = -0.5$$

c. **Update** with $w \leftarrow w - 0.1\,\dfrac{\partial L}{\partial w}$.

$$w_{31} = 0.5 - 0.1(-1) = 0.6$$
$$w_{32} = -0.25 - 0.1(-2) = -0.05$$
$$b_3 = 0.25 - 0.1(-1) = 0.35$$

$$w_{41} = -1 - 0.1(0) = -1$$
$$w_{42} = 0.25 - 0.1(0) = 0.25$$
$$b_4 = -0.5 - 0.1(0) = -0.5$$

$$w_{53} = 2 - 0.1(-0.125) = 2.0125$$
$$w_{54} = 1 - 0.1(0) = 1$$
$$b_5 = -0.5 - 0.1(-0.5) = -0.45$$

New forward pass.

$$z_3 = 0.6(1) - 0.05(2) + 0.35 = 0.85$$
$$a_3 = 0.85$$

$$z_4 = -1(1) + 0.25(2) - 0.5 = -1$$
$$a_4 = 0$$

$$z_5 = 2.0125(0.85) + 1(0) - 0.45 = 1.2606$$
$$y = \sigma(1.2606) = 0.7791$$

$$L = -\ln 0.7791 = 0.2496$$

The loss went from 0.6931 down to 0.2496, so yes, it went down.

d. The three parameters of neuron 4 (w_{41}, w_{42}, b_4) have gradient exactly zero. Neuron 4 had $z_4 = -1$, which is negative, so ReLU gives 0 and its derivative is also 0. That makes $\delta_4 = 0$, and every gradient into neuron 4 is δ_4 times something, so all of them are 0.

The weight w_{54} is also zero, but for a different reason. Its gradient is $\delta_5 a_4$, and $a_4 = 0$.

So neuron 4 was "dead" for this example. It added nothing to the output and it did not learn.

1.3 Sigmoid and Vanishing Gradients

a. Start from $\sigma(z) = \dfrac{1}{1+e^{-z}}$ and differentiate.

$$\begin{aligned}
\sigma'(z) &= \frac{e^{-z}}{(1+e^{-z})^2} \\
&= \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} \\
&= \sigma(z) \cdot \left(1 - \frac{1}{1+e^{-z}}\right) \\
&= \sigma(z)\big(1-\sigma(z)\big)
\end{aligned}$$

Now the values.

$$\begin{aligned}
\sigma'(0) &= 0.5(1-0.5) = 0.2500 \\
\sigma'(2) &= 0.8808(1-0.8808) = 0.1050 \\
\sigma'(5) &= 0.9933(1-0.9933) = 0.0066
\end{aligned}$$

As z gets large, the sigmoid flattens out near 1 and the derivative shrinks toward 0.

b.

$$(0.25)^{10} = 9.5367 \times 10^{-7}$$

That is about one in a million. The first layers get a gradient a million times smaller than it should be, so they learn extremely slowly while the later layers learn normally. This is the vanishing gradient problem.

ReLU helps because its derivative is exactly 1 for any positive input. Multiplying many 1's keeps the gradient the same size, so it can reach the early layers without shrinking.

c. For $n = 256$ the variance is $2/256$, so the standard deviation is its square root.

$$\begin{aligned}
\sigma &= \sqrt{\frac{2}{n}} \\
&= \sqrt{\frac{2}{256}} \\
&= \sqrt{0.0078125} \\
&= 0.0884
\end{aligned}$$

Setting all weights to 0 is a bad idea because every neuron in a layer then computes the exact same thing. They all get the same gradient and so they all get the same update, and they stay identical forever. The layer behaves like a single neuron. Random starting weights break this symmetry. (With ReLU it is even worse: the outputs are 0, so the weight gradients are 0 too and nothing learns.)

1.4 Training Deep Networks: Optimizers and Overfitting

a. Here $f(w) = w^2$, $f'(w) = 2w$, $w_0 = 4$, $\eta = 0.1$.

(i) Plain gradient descent.

$$w_1 = 4 - 0.1(2 \cdot 4) = 4 - 0.8 = 3.2$$
$$w_2 = 3.2 - 0.1(2 \cdot 3.2) = 3.2 - 0.64 = 2.56$$
$$w_3 = 2.56 - 0.1(2 \cdot 2.56) = 2.56 - 0.512 = 2.048$$

(ii) Momentum with $\beta = 0.9$ and $v_0 = 0$.

$$v_1 = 0.9(0) + 2(4) = 8$$
$$w_1 = 4 - 0.1(8) = 3.2$$

$$v_2 = 0.9(8) + 2(3.2) = 7.2 + 6.4 = 13.6$$
$$w_2 = 3.2 - 0.1(13.6) = 1.84$$

$$v_3 = 0.9(13.6) + 2(1.84) = 12.24 + 3.68 = 15.92$$
$$w_3 = 1.84 - 0.1(15.92) = 0.248$$

(iii) Momentum is closer to the minimum after 3 steps (0.248 compared to 2.048). Momentum keeps a running sum of past gradients. When the gradients keep pointing the same way, the velocity builds up and the steps get bigger, so it moves faster.

b. With 10,000 examples and a batch size of 100:

$$\text{updates per epoch} = \frac{10000}{100} = 100$$
$$\text{total updates} = 100 \times 20 = 2000$$

One advantage of mini-batches is that the weights get updated much more often (100 times per epoch instead of once), so training makes progress faster. Each update is also cheaper and needs less memory.

c. The network starts to overfit at about epoch 20. Training accuracy keeps going up (94, 97, 99), but validation accuracy peaks at 86% at epoch 20 and then drops (84, 81). The growing gap between the two means the network is memorizing the training data instead of learning something that generalizes. With early stopping I would keep the weights from epoch 20.

d. Start with the regularized loss and take the derivative.

$$L_{reg} = L + \frac{\lambda}{2} w^2$$
$$\frac{\partial L_{reg}}{\partial w} = \frac{\partial L}{\partial w} + \lambda w$$

Plug this into the update rule.

$$w \leftarrow w - \eta \left(\frac{\partial L}{\partial w} + \lambda w \right)$$
$$= w - \eta\lambda w - \eta \frac{\partial L}{\partial w}$$
$$= (1 - \eta\lambda)\, w - \eta \frac{\partial L}{\partial w}$$

Now the numbers, with $w = 2$, $\partial L/\partial w = 0.5$, $\eta = 0.1$, $\lambda = 0.01$.

$$\begin{aligned} \text{without:} \quad & w = 2 - 0.1(0.5) = 1.95 \\ \text{with:} \quad & w = (1 - 0.1 \cdot 0.01)(2) - 0.1(0.5) \\ & = 0.999(2) - 0.05 \\ & = 1.998 - 0.05 = 1.948 \end{aligned}$$

It is called weight decay because every step first multiplies the weight by $(1 - \eta\lambda)$, which is a little less than 1. So the weight shrinks toward 0 each step, unless the gradient pushes it back.

Assumptions I Made

- I rounded intermediate values to four decimals as I went, so the last digit of a result could differ slightly from a calculation that keeps full precision.
- In 1.2, biases are updated the same way as weights, and the gradients are for the single training example only, so there is no averaging over a batch.
- In 1.3(b), I used the largest possible value of $\sigma'(z)$, which is 0.25, for every layer. So $(0.25)^{10}$ is the largest the scaling can be, not the exact value.
- In 1.3(c), I took n to be the number of inputs to the layer.
- In 1.4(c), accuracy is only recorded every 5 epochs, so "epoch 20" is the best epoch among those listed. The true peak could be anywhere between epochs 15 and 25.